# ATLAS FLOW: EXPLICIT TIMBRE-TRAJECTORY MODELING FOR A NAVIGABLE LATENT-SPACE SYNTHESIZER

**Yuxuan Shi**[1*] **Yifeng Huang**[2*] **Jiangnan Zhang**[3*] **Jinyue Guo**[4] **Jiayi Hu**[5†]
[1] University of Pennsylvania, Philadelphia, USA
[2] Fudan University, Shanghai, China
[3] Shanghai Conservatory of Music, Shanghai, China
[4] RITMO, University of Oslo, Oslo, Norway
[5] TwiddleAI, Shanghai, China
shi10@upenn.edu, yfhuang24@m.fudan.edu.cn, johngoner@icloud.com, jinyue.guo@imv.uio.no, jiayifdzl@gmail.com
[*] Equal contribution. [†] Corresponding author.

## ABSTRACT

We introduce *Atlas Flow*, a real-time interactive timbre-morphing system for MIDI-based music production. Given a fixed MIDI sequence, it renders the sequence while exposing a two-dimensional map of a learned 128-dimensional timbre space, obtained by principal component analysis over training trajectories, with instrument presets as labeled anchors. The user draws a path across this map; the path is read as a sequence of anchor coordinates, and a conditional flow-matching model generates the frame-rate 128-dimensional latent trajectory around them. A causal, PQMF-based decoder in the manner of BRAVE renders audio from that trajectory, with MIDI pitch injected as a FiLM condition at every stage. Because each coordinate returns a trajectory rather than a single latent vector, timbre keeps morphing through a held note, and a path drawn across the map becomes timbre automation of the take.

## 1. INTRODUCTION & RELATED WORK

Learned timbre spaces are well studied: perceptually regularized latent spaces [1], conditioned autoencoders that interpolate in real time [2, 3], latent timbre synthesis and its sound-design practice [4, 5], and preset-space interpolation [6]. Pitch is usually handled by conditioning; CAESynth [3] adds adversarial regularization, which we adopt. RAVE [7] and DDSP [8] showed that neural synthesis can run faster than real time on commodity hardware.

These systems share two limits. First, RAVE-style latents are coarse: each vector summarizes 2048 samples, and BRAVE [9] shows the resulting jitter (spanning 46.4 ms) and loudness and pitch errors shrink as compression decreases. Second, they encode trajectories from existing audio; driven from a control surface, there is none to encode. Latent Terrain [10] maps XY coordinates directly into a RAVE latent space, but like a knob it returns one latent vector per coordinate, so any timbral motion within a note must be performed by hand.

We instead have each coordinate return a *trajectory*. Over a fixed MIDI sequence, the system shows a two-dimensional map of a learned timbre space (the *atlas*) with presets as labeled anchors, and renders the sequence while its timbre follows a pointer, so a path traced across the atlas becomes timbre automation for the take. A *flow predictor* generates these trajectories from atlas coordinates. It is trained with conditional flow matching [11], whose straight-line probability paths permit few-step integration and which has recently been made streamable for audio [12]. A pitch-invariant encoder supplies its training targets, and a MIDI-conditioned decoder renders the result.

## 2. METHOD

### 2.1 Training

Training runs in three phases, all using AMP fp16, gradient clipping at 1.0 and 1000 warm-up updates.

**Phase 1: encoder and decoder, 60k updates.** A shared encoder maps the paired renders $x_A$ at note $n_A$ and $x_B$ at $n_B$ to $z_A$ and $z_B$. The decoder runs twice on $z_A$, against its own note and against the other's, giving $\hat{x}_A = D(z_A, n_A)$ and $\hat{x}_B = D(z_A, n_B)$; each is scored against the corresponding real render, and $z_B$ *is never decoded*. This cross branch is the heart of the design: asking A's timbre to render B's note means any pitch stored in $z$ emerges at the wrong pitch, where the reconstruction loss punishes it — multi-resolution STFT (1.0), a short-window multiband term for transients (0.5), envelope (0.1) and RMS (0.25), averaged over both branches. Two lighter terms escalate that demand. $\mathcal{L}_{\text{tie}}$ (0.1) ties $z_A$ to $z_B$, so pitch may not *vary* with the note; $\mathcal{L}_{\text{adv}}$ (0.05) adds that it may not be *detectable*, a classifier predicting the note from the pooled trajectory through a gradient-reversal layer, learning to find pitch as the encoder learns to remove it [13].

**Phase 2: flow predictor, 100k updates.** Phase 1 yields trajectories only by encoding existing audio, phase 2

© Y. Shi, Y. Huang, J. Zhang, J. Guo, and J. Hu. Licensed under a Creative Commons Attribution 4.0 International License (CC BY 4.0). **Attribution:** Y. Shi, Y. Huang, J. Zhang, J. Guo, and J. Hu, "Atlas Flow: Explicit Timbre-Trajectory Modeling for a Navigable Latent-Space Synthesizer", submitted to *ISMIR LBD*, 2026.

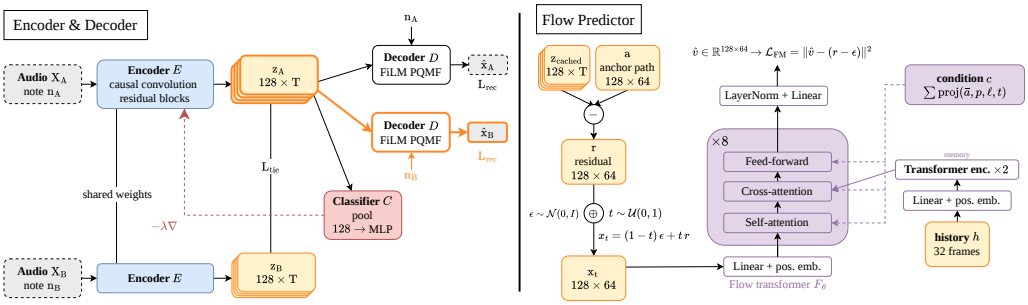



**Figure 1**. Training. **Left:** phase 1, encoder and decoder. **Right:** phase 2, flow predictor.

caches them as targets and learns to produce them from a coordinate. Each preset carries an *anchor* $\bar{a}$, the median across renders of each render's temporal median; an anchor path $a$ repeats it across a 64-frame window, or smoothsteps between two anchors over 0.5 s when the gesture crosses presets. The predictor models only the residual $r = z - a$: the atlas supplies which preset, the network how the timbre moves within it. Windows arrive as full plans (40%, no real history), continuations (40%,) from 32 genuine frames; and a *preset switch* (20%) with history and target from different presets.

The objective is conditional flow matching on that residual. With $\epsilon \sim \mathcal{N}(0, I)$ and $t \sim \mathcal{U}(0, 1)$, the network sees $x_t = (1 - t)\epsilon + tr$ and regresses the velocity $r - \epsilon$ (1.0). Auxiliary terms tie the first generated frame to the last history frame (0.10), match per-channel statistics (0.02) and match first and second temporal differences (0.05). The boundary term earns its keep at inference, where windows are concatenated with no crossfade.

**Phase 3: joint, 30k updates.** Both objectives are summed over all modules, with a further term (0.10) tying the live encoder output to the cached trajectories.

## 2.2 Inference

At inference, the pointer gives two atlas dimensions and a Gaussian-weighted blend of nearby presets gives the other six; the point is then expressed as an inverse-distance mixture of the nearest 128-D anchors, within the connected component of the nearest preset, so every gesture stays a mixture of presets the model has seen and neighbouring coordinates sound alike. Routing from the current position to the target over the mutual 4-NN graph, traversed with a smoothstep $3t^2 - 2t^3$ over 0.5 s, gives the anchor path $a$; its zero endpoint derivatives let a morph start and stop without a discontinuity in rate.

The predictor integrates from noise to $\hat{r}$ with a four-step Heun solver in autoregressive 64-frame blocks, each conditioned on the previous 32 generated frames; the decoder then renders $\hat{z} = a + \hat{r}$ against the MIDI note, which enters the decoder alone.

## 2.3 The trained model

The trained model has three learned parts, 22.2 M parameters in total. The **encoder** (0.63 M) is a causal convolutional stack that maps pitch-normalized features (harmonic magnitudes sampled at multiples of the known $f_0$, plus frame statistics) to a $\tanh$-bounded 128-dimensional trajectory at one frame per 46.4 ms. The **flow predictor** (13.5 M) is a transformer with two layers encoding the history and eight decoder layers, each modulated by adaptive LayerNorm from a per-frame conditioning vector: anchor path, 8-D atlas path, a lifecycle signal (age, gate, release, onset) and flow time. It never sees MIDI, so it cannot encode pitch as timbre, and pitch accuracy rests entirely on the decoder. The **decoder** (7.9 M) is a causal PQMF-based network after BRAVE [9]. Unlike a timbre-transfer decoder, it receives MIDI as a bounded FiLM condition at every residual block rather than estimating pitch from audio, alongside a parameter-free, phase-continuous harmonic excitation built from the note frequency.

## 2.4 A self-contained application

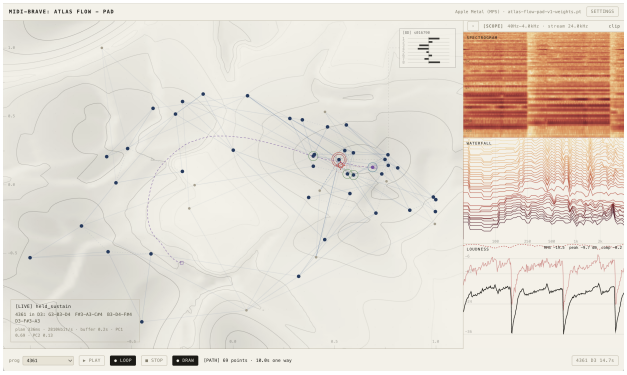


**Figure 2**. The application UI. The atlas contains preset anchors as dots, their nearest-neighbour edges, and shading by how far a request at each point is projected before reaching the model.

The model ships as a self-contained desktop application for macOS and Windows (Figure 2). It runs entirely locally and plays smoothly on a standard CPU, with no GPU or network required. Code and project materials are available at `https://github.com/Twiddle-AI-China/MIDI-Brave`.

## 3. REFERENCES


[1] P. Esling, A. Chemla-Romeu-Santos, and A. Bitton, "Generative timbre spaces: regularizing variational auto-encoders with perceptual metrics," 2018. [Online]. Available: https://arxiv.org/abs/1805.08501

[2] J. T. Colonel and S. Keene, "Conditioning autoencoder latent spaces for real-time timbre interpolation and synthesis," 2020. [Online]. Available: https://arxiv.org/abs/2001.11296

[3] A. V. Puche and S. Lee, "Caesynth: Real-time timbre interpolation and pitch control with conditional autoencoders," 2021. [Online]. Available: https://arxiv.org/abs/2111.05174

[4] K. Tatar, D. Bisig, and P. Pasquier, "Latent timbre synthesis: Audio-based variational auto-encoders for music composition and sound design applications," *Neural Computing and Applications*, vol. 33, no. 1, p. 67–84, Oct. 2020. [Online]. Available: http://dx.doi.org/10.1007/s00521-020-05424-2

[5] K. Tatar, K. Cotton, and D. Bisig, "Sound design strategies for latent audio space explorations using deep learning architectures," 2023. [Online]. Available: https://arxiv.org/abs/2305.15571

[6] G. Le Vaillant and T. Dutoit, "Latent space interpolation of synthesizer parameters using timbre-regularized auto-encoders," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 32, pp. 3379–3392, 2024.

[7] A. Caillon and P. Esling, "Rave: A variational autoencoder for fast and high-quality neural audio synthesis," 2021. [Online]. Available: https://arxiv.org/abs/2111.05011

[8] J. Engel, L. Hantrakul, C. Gu, and A. Roberts, "Ddsp: Differentiable digital signal processing," 2020. [Online]. Available: https://arxiv.org/abs/2001.04643

[9] F. Caspe, J. Shier, M. Sandler, C. Saitis, and A. McPherson, "Designing neural synthesizers for low-latency interaction," 2025. [Online]. Available: https://arxiv.org/abs/2503.11562

[10] S. J. Zheng, K. Yoshida, N. García-Peguinho, J. Liu, D. Hearn, A. Xambó Sedó, and N. Bryan-Kinns, "Latent terrain: Adapting neural audio autoencoders as design materials in nime," in *Proceedings of the International Conference on New Interfaces for Musical Expression*, London, UK, 2026.

[11] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le, "Flow matching for generative modeling," 2023. [Online]. Available: https://arxiv.org/abs/2210.02747

[12] S. Liang, D. Markovic, I. D. Gebru, S. Krenn, T. Keebler, J. Sandakly, F. Yu, S. Hassel, C. Xu, and A. Richard, "Binauralflow: A causal and streamable approach for high-quality binaural speech synthesis with flow matching models," 2025. [Online]. Available: https://arxiv.org/abs/2505.22865

[13] Y. Ganin and V. Lempitsky, "Unsupervised domain adaptation by backpropagation," 2015. [Online]. Available: https://arxiv.org/abs/1409.7495